

Capítulo 5: Estimação de Parâmetros

Bioinformática para Biologia de Sistemas

Ney Lemke

DBF-Unesp

12 de janeiro de 2026

Sumário

- 1 Introdução
- 2 Formulação do Problema
- 3 Busca em Grade (Grid Search)
- 4 Métodos de Gradiente
- 5 Algoritmos Globais
- 6 Análise de Identificabilidade
- 7 Validação e Critérios de Ajuste
- 8 Exercícios
- 9 Referências

5.1 Por que Estimação de Parâmetros?

Objetivos de Aprendizagem

- Compreender o problema inverso em biologia de sistemas
- Dominar métodos de otimização local e global
- Aplicar técnicas de estimação a modelos biológicos
- Avaliar identificabilidade e incerteza de parâmetros
- Validar modelos com critérios estatísticos

Definição: Estimação de Parâmetros

Processo de determinar valores numéricos de parâmetros de um modelo matemático a partir de dados experimentais, ajustando o modelo para reproduzir o comportamento observado do sistema.

O Problema Inverso

Problema Direto:

- Parâmetros conhecidos
- Condições iniciais dadas
- Simular $\rightarrow$ Previsões
- Determinístico

Exemplo

Dados $k_1 = 0.5$, $k_2 = 0.1$

Simular: $\frac{dx}{dt} = k_1 - k_2x$

$\Rightarrow$ obter $x(t)$

Problema Inverso:

- Dados experimentais
- Parâmetros desconhecidos
- Estimar $\leftarrow$ Ajuste
- Mal-posto (ill-posed)

Exemplo

Dados $x(t)$ experimentais

Modelo: $\frac{dx}{dt} = k_1 - k_2x$

$\Rightarrow$ encontrar k_1 , k_2

Importante!

O problema inverso é geralmente mais difícil que o direto

Desafios na Estimação de Parâmetros

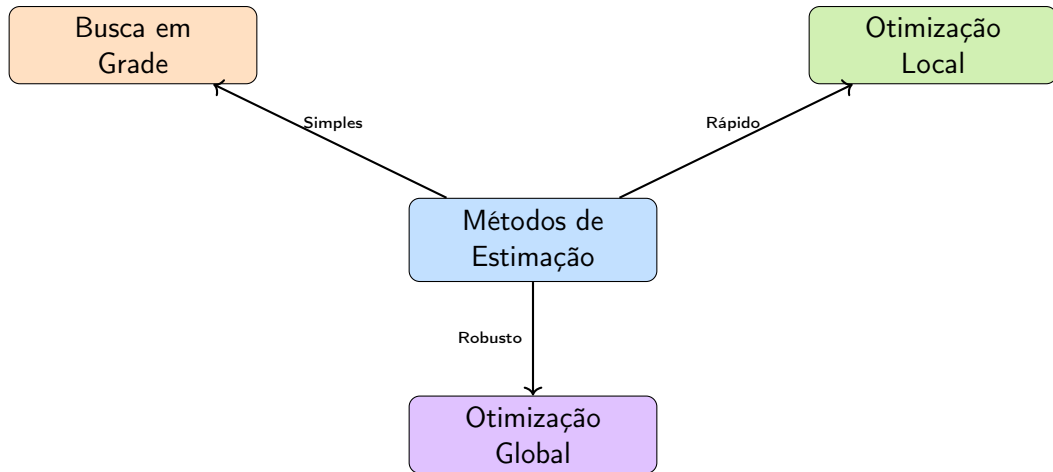
Principais Obstáculos

1. **Identificabilidade:** múltiplas soluções possíveis
2. **Ruído experimental:** medições imprecisas
3. **Dados escassos:** poucas observações
4. **Mínimos locais:** otimização não-convexa
5. **Correlação de parâmetros:** dificuldade de separação
6. **Custo computacional:** simulações repetidas
7. **Incerteza:** quantificação de confiança

$J(\theta)$



Visão Geral dos Métodos



Escolha do Método

Função Objetivo

Definição: Função Objetivo (Cost Function)

Medida quantitativa da discrepância entre dados experimentais e previsões do modelo.
Objetivo: minimizar esta função.

Componentes

- y_i : dados experimentais (observações)
- $\hat{y}_i(\boldsymbol{\theta})$: previsões do modelo
- $\boldsymbol{\theta}$: vetor de parâmetros a estimar
- n : número de observações

$$J(\boldsymbol{\theta}) = \text{medida de erro entre } \{y_i\} \text{ e } \{\hat{y}_i(\boldsymbol{\theta})\}$$

Mínimos Quadrados Ordinários (OLS)

Método Mais Comum

Minimizar a soma dos quadrados dos resíduos:

$$J(\theta) = \text{SSE}(\theta) = \sum_{i=1}^n [y_i - \hat{y}_i(\theta)]^2$$

Vantagens:

- Simples e intuitivo
- Bem estabelecido
- Solução analítica (modelos lineares)
- Interpretação estatística

Suposições:

- Erros gaussianos
- Variância constante
- Erros independentes
- Outliers problemáticos

Mínimos Quadrados Ponderados (WLS)

Quando usar?

Quando as observações têm diferentes níveis de incerteza ou confiabilidade

$$J(\boldsymbol{\theta}) = \sum_{i=1}^n w_i [y_i - \hat{y}_i(\boldsymbol{\theta})]^2$$

Pesos w_i :

- $w_i = 1/\sigma_i^2$: inverso da variância (comum)
- $w_i = 1/y_i$: proporcional ao valor (dados log-scale)
- w_i customizado: baseado em conhecimento experimental

Exemplo: Cinética Enzimática

Maximum Likelihood Estimation (MLE)

Abordagem Estatística

Encontrar parâmetros que maximizam a probabilidade (likelihood) de observar os dados

Função de Verossimilhança:

$$\mathcal{L}(\boldsymbol{\theta}) = P(\text{dados}|\boldsymbol{\theta})$$

Log-verossimilhança (mais conveniente):

$$\ln \mathcal{L}(\boldsymbol{\theta}) = \sum_{i=1}^n \ln P(y_i|\boldsymbol{\theta})$$

Caso Gaussiano

Se erros $\sim \mathcal{N}(0, \sigma^2)$:

Abordagem Bayesiana

Incorpora Conhecimento Prévio

Usa teorema de Bayes para atualizar crenças sobre parâmetros

$$P(\theta|\text{dados}) = \frac{P(\text{dados}|\theta) \cdot P(\theta)}{P(\text{dados})}$$

- $P(\theta|\text{dados})$: **posterior** (o que queremos)
- $P(\text{dados}|\theta)$: **likelihood** (ajuste aos dados)
- $P(\theta)$: **prior** (conhecimento prévio)
- $P(\text{dados})$: constante de normalização

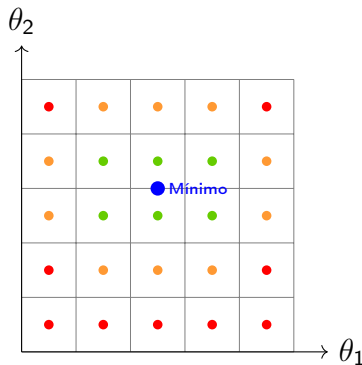
Vantagens

- Incorpora informação da literatura

Conceito de Busca em Grade

Definição: Grid Search

Método de otimização que avalia sistematicamente a função objetivo em uma grade regular de valores de parâmetros.



Vantagens e Desvantagens

Vantagens

- Simples de implementar
- Facilmente paralelizável
- Garantia de exploração
- Visualização intuitiva
- Sem derivadas necessárias

Desvantagens

- Custo exponencial: $O(n^P)$
- Inviável para muitos parâmetros
- Resolução limitada
- Desperdício de avaliações
- Maldição da dimensionalidade

Importante!

Custo Computacional:

Com 10 valores por parâmetro:

- 2 parâmetros: $10^2 = 100$ avaliações
- 5 parâmetros: $10^5 = 100.000$ avaliações

Implementação: Grid Search 1D

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def objective_function(theta):
5     """Funcao objetivo a minimizar"""
6     return (theta - 3)**2 + 5
7
8 # Definir grade
9 theta_values = np.linspace(0, 6, 50)
10
11 # Avaliar funcao objetivo
12 J_values = [objective_function(theta) for theta in theta_values]
13
14 # Encontrar minimo
15 idx_min = np.argmin(J_values)
16 theta_best = theta_values[idx_min]
17 J_best = J_values[idx_min]
```

Implementação: Grid Search 2D

```
1 def sse_model(params, t_data, y_data):
2     """SSE para modelo exponencial  $y = a \cdot \exp(-b \cdot t)$ """
3     a, b = params
4     y_pred = a * np.exp(-b * t_data)
5     sse = np.sum((y_data - y_pred)**2)
6     return sse
7
8 # Dados experimentais
9 t_data = np.array([0, 1, 2, 3, 4, 5])
10 y_data = np.array([10, 6.1, 3.7, 2.2, 1.4, 0.8])
11
12 # Grade de parametros
13 a_range = np.linspace(5, 15, 20)
14 b_range = np.linspace(0.1, 1.0, 20)
15
16 # Avaliar em toda a grade
17 SSE_grid = np.zeros((len(a_range), len(b_range)))
```

Visualização: Contour Plot

```
1 # Criar contour plot
2 A, B = np.meshgrid(a_range, b_range)
3
4 plt.figure(figsize=(10, 8))
5 contour = plt.contourf(A, B, SSE_grid.T, levels=20,
6                        cmap='viridis')
7 plt.colorbar(contour, label='SSE')
8
9 # Marcar minimo
10 plt.plot(a_best, b_best, 'r*', markersize=20,
11          label=f'Minimo: ({a_best:.2f}, {b_best:.2f})')
12
13 # Contornos
14 plt.contour(A, B, SSE_grid.T, levels=10,
15            colors='white', linewidths=0.5, alpha=0.5)
16
17 plt.xlabel('Parametro a')
```

Gradient Descent (Descida do Gradiente)

Definição: Gradient Descent

Método iterativo que se move na direção do negativo do gradiente para encontrar um mínimo local da função objetivo.

$$\theta^{(k+1)} = \theta^{(k)} - \alpha \nabla J(\theta^{(k)})$$

- $\theta^{(k)}$: parâmetros na iteração k
- α : taxa de aprendizado (step size)
- $\nabla J = \left[\frac{\partial J}{\partial \theta_1}, \frac{\partial J}{\partial \theta_2}, \dots \right]^T$: gradiente

J
↑

Início

Escolha da Taxa de Aprendizado

Importante!

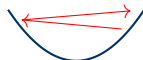
A escolha de α é crucial para convergência!



α pequeno
Lento



α ideal
Rápido



α grande
Oscila

Estratégias

- **Fixo:** α constante (simples, pode não convergir)
- **Decrescente:** $\alpha_k = \alpha_0 / (1 + k)$ (garante convergência)
- **Line search:** otimizar α em cada iteração (mais robusto)
- **Adaptativo:** ajustar baseado em progresso (Adam, RMSprop)

Critérios de Convergência

Quando Parar?

Teste múltiplos critérios simultaneamente:

1. **Gradiente pequeno:** $\|\nabla J(\boldsymbol{\theta}^{(k)})\| < \epsilon_g$
 - Indica ponto estacionário
2. **Mudança em parâmetros:** $\|\boldsymbol{\theta}^{(k+1)} - \boldsymbol{\theta}^{(k)}\| < \epsilon_\theta$
 - Parâmetros não mudam mais
3. **Mudança na função:** $|J^{(k+1)} - J^{(k)}| < \epsilon_J$
 - Função objetivo estabilizou
4. **Número máximo de iterações:** $k > k_{\max}$
 - Prevenir loops infinitos

Valores Típicos

$$\epsilon_g = 10^{-6}, \epsilon_\theta = 10^{-6}, \epsilon_J = 10^{-8}, k_{\max} = 1000$$

Implementação: Gradient Descent

```
1 def gradient_descent(f, theta0, alpha=0.01, max_iter=1000,
2                       tol=1e-6):
3     """
4     Gradient descent com derivadas numericas
5     f: funcao objetivo
6     theta0: chute inicial
7     """
8     theta = np.array(theta0, dtype=float)
9     history = [theta.copy()]
10
11    for k in range(max_iter):
12        # Calcular gradiente numericamente
13        grad = numerical_gradient(f, theta)
14
15        # Atualizar parametros
16        theta_new = theta - alpha * grad
17
```

Método de Newton-Raphson

Ideia

Usar informação de segunda ordem (curvatura) para convergência mais rápida

$$\theta^{(k+1)} = \theta^{(k)} - H^{-1}(\theta^{(k)}) \nabla J(\theta^{(k)})$$

- H: matriz Hessiana (derivadas segundas)
- $H_{ij} = \frac{\partial^2 J}{\partial \theta_i \partial \theta_j}$

Vantagens:

- Convergência quadrática
- Muito rápido perto do mínimo
- Não precisa de step size

Desvantagens:

- Calcular H é caro
- Inverter H também
- Pode não convergir longe do mínimo
- Requer H positivo definida

Método de Gauss-Newton

Específico para Mínimos Quadrados

Aproxima Hessiana usando Jacobiano das previsões do modelo

Para $J(\boldsymbol{\theta}) = \sum_{i=1}^n r_i^2(\boldsymbol{\theta})$ onde $r_i = y_i - \hat{y}_i(\boldsymbol{\theta})$:

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} + (\mathbf{J}^T \mathbf{J})^{-1} \mathbf{J}^T \mathbf{r}$$

- $\mathbf{J}$: matriz Jacobiana dos resíduos ($J_{ij} = \partial r_i / \partial \theta_j$)
- $\mathbf{r}$: vetor de resíduos

Vantagens

- Evita calcular Hessiana completa
- Boa performance para problemas de mínimos quadrados

Levenberg-Marquardt (LM)

Definição: Levenberg-Marquardt

Combina Gauss-Newton e gradient descent, interpolando entre ambos via parâmetro de amortecimento.

$$\theta^{(k+1)} = \theta^{(k)} + (J^T J + \lambda I)^{-1} J^T r$$

- λ : parâmetro de amortecimento (damping)
- λ grande $\Rightarrow$ gradient descent (pequenos passos, robusto)
- λ pequeno $\Rightarrow$ Gauss-Newton (passos grandes, rápido)

Estratégia Adaptativa:

- Se J diminui: aceitar passo, diminuir λ (mais agressivo)
- Se J aumenta: rejeitar passo, aumentar λ (mais conservador)

Exemplo: Michaelis-Menten com scipy

```
1 from scipy.optimize import least_squares, curve_fit
2 import numpy as np
3
4 # Modelo de Michaelis-Menten
5 def michaelis_menten(S, Vmax, Km):
6     return Vmax * S / (Km + S)
7
8 # Dados experimentais
9 S_data = np.array([0.5, 1, 2, 5, 10, 20, 50])
10 v_data = np.array([0.4, 0.7, 1.2, 1.8, 2.1, 2.3, 2.4])
11
12 # Metodo 1: curve_fit (interface simples)
13 params, cov = curve_fit(michaelis_menten, S_data, v_data,
14                          p0=[3, 5])
15 Vmax_fit, Km_fit = params
16
17 print(f"Vmax = {Vmax_fit:.3f}")
```

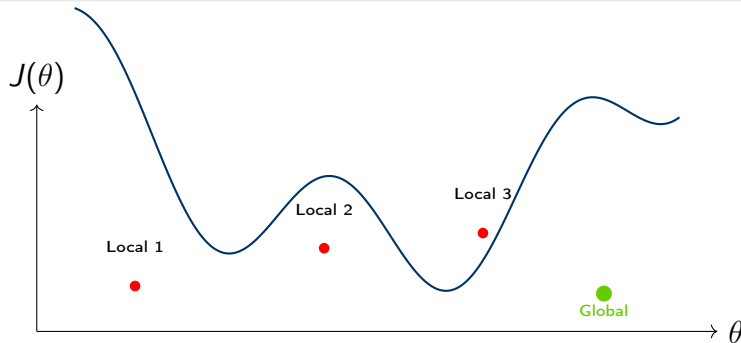
Visualização do Ajuste

```
1 # Gerar curva do modelo ajustado
2 S_smooth = np.linspace(0, 60, 200)
3 v_fit = michaelis_menten(S_smooth, Vmax_fit, Km_fit)
4
5 # Plotar
6 plt.figure(figsize=(10, 6))
7 plt.plot(S_data, v_data, 'ko', markersize=10,
8          label='Dados experimentais')
9 plt.plot(S_smooth, v_fit, 'b-', linewidth=2,
10          label=f'Ajuste: Vmax={Vmax_fit:.2f}, Km={Km_fit:.2f}')
11
12 # Linhas de referencia
13 plt.axhline(y=Vmax_fit, color='r', linestyle='--',
14             alpha=0.5, label=f'Vmax')
15 plt.axvline(x=Km_fit, color='g', linestyle='--',
16             alpha=0.5, label=f'Km')
17 plt.axhline(y=Vmax_fit/2, color='gray', linestyle=':')
```

Por que Otimização Global?

Problema

Métodos locais podem ficar presos em mínimos locais



Quando Usar Otimização Global?

- Função objetivo altamente não-linear

Algoritmos Genéticos (GA)

Definição: Algoritmo Genético

Método inspirado na evolução biológica que mantém uma população de soluções candidatas e as evolui usando seleção, cruzamento e mutação.

Analogia Biológica:

- **Indivíduo:** conjunto de parâmetros θ
- **Gene:** parâmetro individual θ_i
- **Fitness:** inverso da função objetivo $1/J(\theta)$
- **População:** conjunto de soluções candidatas

Operadores Genéticos

1. **Seleção:** escolher indivíduos com melhor fitness
2. **Cruzamento:** combinar dois pais para gerar filhos
3. **Mutação:** introduzir variação aleatória

Simulated Annealing (SA)

Inspiração

Processo de recozimento em metalurgia: aquecer metal e esfriar lentamente para alcançar estrutura cristalina de mínima energia

Ideia Principal:

- Permitir movimentos para soluções piores com probabilidade decrescente
- "Temperatura" controla aceitação de soluções piores
- Alta temperatura: exploração ampla
- Baixa temperatura: refinamento local

$$P(\text{aceitar}) = \begin{cases} 1 & \text{se } \Delta J < 0 \\ e^{-\Delta J/T} & \text{se } \Delta J \geq 0 \end{cases}$$

Differential Evolution (DE)

Definição: Evolução Diferencial

Algoritmo evolutivo que cria novos candidatos através de diferenças entre membros da população.

Operação Principal:

1. Selecionar 3 indivíduos aleatórios: $\mathbf{a}$, $\mathbf{b}$, $\mathbf{c}$
2. Criar vetor de mutação: $\mathbf{v} = \mathbf{a} + F(\mathbf{b} - \mathbf{c})$
3. Cruzamento: $\mathbf{u}_i = \begin{cases} v_i & \text{se } rand < CR \\ \theta_i & \text{caso contrário} \end{cases}$
4. Seleção: manter melhor entre atual e novo
 - F : fator de escala (differential weight), tipicamente $F \in [0.5, 1]$
 - CR : taxa de cruzamento (crossover rate), tipicamente $CR \in [0.5, 0.9]$

Vantagens

Particle Swarm Optimization (PSO)

Inspiração

Comportamento social de bandos de pássaros ou cardumes de peixes

Conceito:

- Cada partícula tem posição $\mathbf{x}_i$ e velocidade $\mathbf{v}_i$
- Partículas se movem influenciadas por:
 - Sua melhor posição histórica ($\mathbf{p}_i$)
 - Melhor posição global do enxame ($\mathbf{g}$)

$$\begin{aligned}\mathbf{v}_i^{(k+1)} &= w\mathbf{v}_i^{(k)} + c_1r_1(\mathbf{p}_i - \mathbf{x}_i^{(k)}) + c_2r_2(\mathbf{g} - \mathbf{x}_i^{(k)}) \\ \mathbf{x}_i^{(k+1)} &= \mathbf{x}_i^{(k)} + \mathbf{v}_i^{(k+1)}\end{aligned}$$

- w : peso de inércia

Implementação: Differential Evolution

```
1 from scipy.optimize import differential_evolution
2
3 # Funcao objetivo
4 def objective(params, t_data, y_data):
5     """Modelo exponencial:  $y = a * \exp(-b*t)$ """
6     a, b = params
7     y_pred = a * np.exp(-b * t_data)
8     sse = np.sum((y_data - y_pred)**2)
9     return sse
10
11 # Dados experimentais
12 t_data = np.array([0, 1, 2, 3, 4, 5])
13 y_data = np.array([10, 6.1, 3.7, 2.2, 1.4, 0.8])
14
15 # Definir bounds para parametros
16 bounds = [(1, 20), (0.01, 2.0)] # [(a_min, a_max), (b_min, b_max)]
```

Comparação de Métodos

Método	Velocidade	Global	Derivadas	Parâmetros	Uso
Grid Search	—	Sim	Não	Poucos	Exploração
Gradient Descent	++	Não	Sim	Média	Local
Newton-Raphson	+++	Não	2ª ordem	Média	Refinamento
Levenberg-Marquardt	+++	Não	Jacobiano	Média	Mín. Quad.
GA	+	Sim	Não	Muitos	Robusto
Simulated Annealing	+	Sim	Não	Muitos	Explora
Differential Evolution	++	Sim	Não	Média	Melhor
Particle Swarm	++	Sim	Não	Média	Rápido

Recomendações

Identificabilidade Estrutural

Definição: Identificabilidade Estrutural

Propriedade teórica do modelo: os parâmetros podem ser determinados unicamente a partir de dados perfeitos (infinitos e sem ruído)?

Identificável

$$\frac{dx}{dt} = k_1 - k_2 x$$

Ambos k_1 e k_2 podem ser determinados unicamente

Não Identificável

$$\frac{dx}{dt} = (k_1 k_2) - k_2 x$$

Apenas o produto $k_1 k_2$ é identificável, não k_1 e k_2 separadamente

Importante!

Antes de estimar parâmetros, verifique se o modelo é estruturalmente identificável!

Identificabilidade Prática

Definição: Identificabilidade Prática

Podem os parâmetros ser determinados com precisão razoável a partir de dados experimentais reais (finitos e com ruído)?

Causas de Não-Identificabilidade Prática:

- Dados insuficientes ou de baixa qualidade
- Parâmetros altamente correlacionados
- Modelo super-parametrizado (overfitting)
- Sensibilidade muito baixa a alguns parâmetros

Exemplo: Correlação de Parâmetros

Modelo: $y = a \cdot e^{-b \cdot t}$

Se t é pequeno: $e^{-bt} \approx 1 - bt$

Matriz de Informação de Fisher (FIM)

Quantifica Informação sobre Parâmetros

Mede quanta informação os dados contêm sobre cada parâmetro

$$F_{ij} = \sum_{k=1}^n \frac{1}{\sigma_k^2} \frac{\partial \hat{y}_k}{\partial \theta_i} \frac{\partial \hat{y}_k}{\partial \theta_j}$$

onde:

- $\hat{y}_k$: previsão do modelo no ponto k
- σ_k^2 : variância do erro no ponto k
- θ_i, θ_j : parâmetros

Interpretação:

- FIM grande $\Rightarrow$ alta informação, parâmetros bem determinados

Intervalos de Confiança

Estimativa Assintótica

Para grandes amostras, $\theta \sim \mathcal{N}(\theta^*, C)$ onde:

$$C = \text{Cov}(\theta) \approx F^{-1}$$

Intervalo de Confiança (95%):

$$\theta_i^* \pm 1.96 \sqrt{C_{ii}}$$

Na Prática

`scipy.optimize.curve_fit` retorna matriz de covariância:

- Diagonal: $\sqrt{C_{ii}}$ = erro padrão do parâmetro i
- Off-diagonal: C_{ij} = correlação entre parâmetros i e j

Profile Likelihood

Definição: Profile Likelihood

Método robusto para calcular intervalos de confiança explorando o espaço de parâmetros sistematicamente.

Procedimento para parâmetro θ_i :

1. Fixar θ_i em valor específico
2. Otimizar todos os outros parâmetros
3. Registrar $J_{\min}(\theta_i)$
4. Repetir para vários valores de θ_i
5. Plotar $J_{\min}$ vs θ_i



Análise de Correlação de Parâmetros

```
1 from scipy.optimize import curve_fit
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4
5 # Ajustar modelo
6 params, cov = curve_fit(model_func, x_data, y_data)
7
8 # Calcular matriz de correlacao
9 std = np.sqrt(np.diag(cov))
10 corr = cov / np.outer(std, std)
11
12 # Visualizar
13 param_names = ['k1', 'k2', 'k3']
14 plt.figure(figsize=(8, 6))
15 sns.heatmap(corr, annot=True, fmt='.2f',
16             xticklabels=param_names,
17             yticklabels=param_names,
```

Implementação: Intervalos de Confiança

```
1 # Extrair parametros e erros
2 params_best = params
3 errors = np.sqrt(np.diag(cov))
4
5 # Intervalo de confianca 95% (aproximacao normal)
6 alpha = 0.05
7 from scipy import stats
8 t_val = stats.t.ppf(1-alpha/2, len(x_data)-len(params))
9 ci = t_val * errors
10
11 # Visualizar
12 fig, ax = plt.subplots(figsize=(10, 6))
13 x_pos = np.arange(len(param_names))
14
15 ax.bar(x_pos, params_best, yerr=ci, alpha=0.7,
16        capsize=10, color='steelblue',
17        error_kw={'elinewidth': 2, 'capthick': 2})
```

Análise de Resíduos

Verificação de Hipóteses

Examinar resíduos $r_i = y_i - \hat{y}_i$ para validar suposições do modelo

Padrões a Verificar:

Bom Ajuste

- Distribuição aleatória
- Sem padrão sistemático
- Centrados em zero
- Variância constante
- Aproximadamente normais

Problemas

- Padrão curvo: modelo inadequado
- Funil: heterocedasticidade
- Outliers: dados anômalos
- Não-normalidade: transformar dados

Coeficiente de Determinação (R^2)

Definição: R^2

Proporção da variância nos dados explicada pelo modelo. Varia de 0 (nenhum ajuste) a 1 (ajuste perfeito).

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

- SS_{res} : soma dos quadrados dos resíduos
- SS_{tot} : soma dos quadrados totais
- $\bar{y}$: média dos dados

ATENÇÃO!

Penaliza Complexidade do Modelo

$$R^2_{\text{adj}} = 1 - \frac{(1 - R^2)(n - 1)}{n - p - 1}$$

onde:

- n : número de observações
- p : número de parâmetros

Características:

- Pode diminuir ao adicionar parâmetros ruins
- Mais apropriado para comparação de modelos
- Ainda limitado como métrica única

Akaike Information Criterion (AIC)

Definição: AIC

Critério que balanceia qualidade do ajuste com complexidade do modelo, baseado em teoria da informação.

$$\text{AIC} = 2p - 2 \ln(\mathcal{L}) \approx n \ln \left(\frac{\text{SSE}}{n} \right) + 2p$$

- p : número de parâmetros
- $\mathcal{L}$: máxima verossimilhança
- SSE: soma dos quadrados dos erros

Interpretação:

- Menor AIC = melhor modelo
- Penaliza adição de parâmetros

Bayesian Information Criterion (BIC)

Definição: BIC (ou Schwarz Criterion)

Similar ao AIC, mas penaliza mais fortemente a complexidade do modelo.

$$\text{BIC} = p \ln(n) - 2 \ln(\mathcal{L}) \approx n \ln \left(\frac{\text{SSE}}{n} \right) + p \ln(n)$$

Comparação AIC vs BIC:

- BIC penaliza mais complexidade que AIC (termo $\ln(n)$ vs 2)
- BIC tende a selecionar modelos mais simples
- AIC minimiza erro de previsão; BIC busca "modelo verdadeiro"
- Para $n > 7$: BIC penaliza mais que AIC

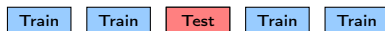
Cross-Validation (Validação Cruzada)

Teste de Capacidade Preditiva

Avaliar desempenho do modelo em dados não vistos durante o treinamento

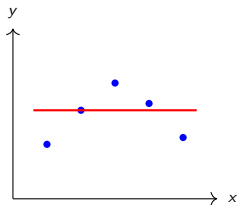
K-Fold Cross-Validation:

1. Dividir dados em K subconjuntos (folds)
2. Para cada fold k :
 - Treinar modelo nos outros $K - 1$ folds
 - Testar no fold k
 - Calcular erro de previsão
3. Calcular erro médio de CV

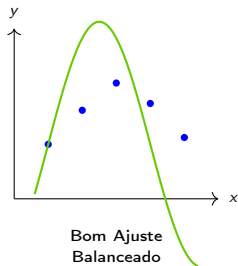


Fold 3 como teste

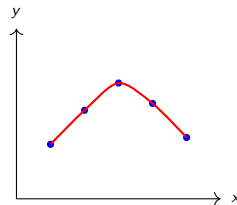
Overfitting vs Underfitting



Underfitting
Viés alto



Bom Ajuste
Balanceado



Overfitting
Variância alta

Sintomas de Overfitting

- Erro de treinamento muito menor que erro de teste
- Modelo complexo com muitos parâmetros
- Intervalos de confiança muito largos
- Previsões ruins fora do range dos dados

Implementação: Cálculo de Métricas

```
1 import numpy as np
2
3 def calculate_metrics(y_true, y_pred, n_params):
4     """Calcula varias metricas de ajuste"""
5     n = len(y_true)
6
7     # Residuos
8     residuals = y_true - y_pred
9
10    # SSE e MSE
11    sse = np.sum(residuals**2)
12    mse = sse / n
13    rmse = np.sqrt(mse)
14
15    # R-squared
16    ss_tot = np.sum((y_true - np.mean(y_true))**2)
17    r2 = 1 - (sse / ss_tot)
```

Exercícios - Parte 1

Questão 1: Busca em Grade

Implemente busca em grade para encontrar parâmetros a e b do modelo $y = a \cdot x^b$ usando os dados:

x	1	2	3	4	5	6
y	2.1	8.3	18.2	32.5	50.1	72.3

Use: $a \in [1, 3]$, $b \in [1.5, 2.5]$, grade 20x20.

Questão 2: Gradient Descent

Implemente gradient descent manualmente para minimizar $J(\theta) = (\theta - 5)^2 + 3$:

1. Use $\theta_0 = 0$ e $\alpha = 0.1$
2. Plote trajetória de convergência
3. Teste diferentes valores de α (0.01, 0.1, 0.5, 1.0)

Exercícios - Parte 2

Questão 3: Levenberg-Marquardt

Ajuste parâmetros da cinética de Michaelis-Menten $v = \frac{V_{\max}[S]}{K_M + [S]}$ aos dados:

[S] (mM)	0.1	0.5	1	2	5	10	20
v ($\mu\text{mol/min}$)	0.3	1.1	1.8	2.5	3.3	3.7	3.9

1. Use `scipy.optimize.curve_fit`
2. Calcule intervalos de confiança
3. Plote dados vs modelo ajustado
4. Faça análise de resíduos

Questão 4: Differential Evolution

Compare DE com LM no problema anterior:

Exercícios - Parte 3

Questão 5: Modelo de Crescimento Logístico

Considere dados de crescimento bacteriano:

1. Gere dados sintéticos: $N(t) = \frac{K}{1 + \left(\frac{K}{N_0} - 1\right)e^{-rt}}$
com $N_0 = 1$, $r = 0.5$, $K = 100$, $t \in [0, 20]$, adicione ruído gaussiano
2. Estime r e K usando três métodos diferentes
3. Compare SSE, AIC e tempo computacional
4. Analise identificabilidade: r e K são correlacionados?
5. Calcule profile likelihood para r e K

Questão 6: Validação Cruzada

Implemente 5-fold cross-validation para o modelo logístico:

1. Divida dados em 5 folds

Projeto: Sistema de Lotka-Volterra

Estime parâmetros do sistema predador-presa:

$$\begin{aligned}\frac{dV}{dt} &= \alpha V - \beta VP \\ \frac{dP}{dt} &= \delta \beta VP - \gamma P\end{aligned}$$

Dados simulados: Gere séries temporais com parâmetros conhecidos + ruído

Tarefas:

1. Implementar função de custo (SSE de ambas variáveis)
2. Estimar 4 parâmetros usando Differential Evolution
3. Refinar com Levenberg-Marquardt
4. Analisar identificabilidade (matriz de correlação)

Referências Principais

-  Nocedal J., Wright S.J. (2006). *Numerical Optimization*, 2nd ed. Springer.
-  Raue A. et al. (2009). Structural and practical identifiability analysis of partially observed dynamical models. *Bioinformatics*.
-  Voit E.O. et al. (2015). 150 years of the mass action law. *PLoS Computational Biology*.
-  Moles C.G. et al. (2003). Parameter estimation in biochemical pathways. *Genome Research*.
-  Burnham K.P., Anderson D.R. (2002). *Model Selection and Multimodel Inference*, 2nd ed. Springer.

Livros

- Press W.H. et al. (2007). *Numerical Recipes*, 3rd ed. Cambridge.
- Bard Y. (1974). *Nonlinear Parameter Estimation*. Academic Press.
- Seber G.A.F., Wild C.J. (2003). *Nonlinear Regression*. Wiley.

Artigos Importantes

- Gutenkunst R.N. et al. (2007). Universally sloppy parameter sensitivities. *PLoS Comp. Biol.*
- Villaverde A.F. et al. (2016). A cooperative strategy for parameter estimation. *BMC Sys. Biol.*
- Hug S. et al. (2013). High-dimensional Bayesian parameter estimation. *Math. Biosci.*

Obrigado!

Capítulo 5: Estimação de Parâmetros

Próximo Capítulo:
Sistemas Gênicos

Dúvidas? Perguntas?